Project Proposal Milestone 0: BeerChat

Course: CS 415/515 – Software Engineering

Submission Date: September 2, 2026

Git Tag: milestone-0

Repository:https://github.com/Arcce9/BeerChat

1. Team & Project Details

Project Name: BeerChat

Members:

- Arda Alici aalici@crimson.ua.edu — Frontend, Maps, UI styling
- Margulan Baizhakyp mbaizhakyp@crimson.ua.edu — Supabase backend, Auth, Realtime database
- Karthik Gaur kgaur@crimson.ua.edu — Matching logic, OpenAI embeddings, Testing & CI

2. Executive Summary

Going out to local bars often leaves people feeling disconnected despite being surrounded by crowds. Most existing social apps focus on long-distance matching or planning events days in advance rather than spontaneous, real-time local connections. BeerChat solves this by offering a venue-based presence platform. Users select a venue they plan to visit, see who else is there and open to chatting, and mutually connect through brief, temporary text messages to meet up in person.

3. Target Users & Personas

Target Audience

- **Primary:** College students and young adults (ages 18–30) spending time at nightlife venues.
- **Secondary:** Travelers and newcomers looking to meet locals on the fly.

Personas

- **Alex (21, Student):** Wants to find people to talk to about shared interests (like board games or startups) while out at a local spot, but dislikes awkward cold approaches and values personal privacy.
- **Sam (26, Software Engineer):** Recently moved to town and wants to make local friends spontaneously without spending days texting back and forth on dating or social apps.

4. Proposed Application

- **What it does:** Facilitates quick, safe in-person meetups at specific venues.
- **Tech format:** iOS mobile app built with React Native and Expo.
- **Design direction:** Dark theme tailored for dark venues, featuring large touch targets (>44 pt) so it is easy to use with one hand.
- **User flow:** Sign up Set name & interests → View map → Tap venue→ Check in ("I'm here") → Browse present users→ Tap "Open to meet" → Double yes match → Chat & meet up.

5. MVP Scope (Milestone 1)

Included in MVP

- Email/password authentication via Supabase.
- Minimal profile (first name + 3 to 5 interest tags).
- Apple Maps view pre-loaded with local seed venues.
- Status updates ("Heading there" / "I'm here") with an explicit discoverability toggle.
- Interest-ranked suggestion list of users at the same bar.
- Double opt-in matching ("Open to meet?").
- Real-time 1-on-1 chat powered by Supabase.

Out of Scope

- Background location tracking or sharing exact GPS coordinates.
- Bios, follower counts, photo feeds, or public profiles.
- Group chats, venue ratings, or reviews.

6. System Requirements

Priority User Stories

1. As a user, I want to check into a bar so that others at the venue know I am open to meeting up.
    - *Acceptance Criteria:* Users do not appear in the venue list until they explicitly check in and toggle visibility on.
2. As a user, I want connections to require both people to agree so that I am not bothered by unwanted messages.
    - *Acceptance Criteria:* Expressing interest remains completely private until the second person also opts in.

Key Functional Requirements

FR1: The app must only display users checked into the exact same venue.

FR2: Chat messages must transmit instantly without requiring a page refresh.

Non-Functional Requirements

- Privacy: Exact GPS coordinates are never sent across the network or stored in the database.
- Usability: Standardized dark mode optimized for low-light environments.
- Speed: Live presence updates delivered in under 500ms.

7. Architecture & Stack

- Architecture: Client-Server model using React Native for the front end and cloud API services on the back end.
- Frontend: React Native, Expo, TypeScript.
- Backend: Supabase (Auth, Postgres, Realtime listeners).
- Live Services: Node.js + Socket.io (added in Step 6 for presence).
- AI & Ranking: OpenAI text-embedding-3-small stored in Postgres pgvector to rank shared interests.
- Maps**:** react-native-maps running Apple Maps natively.

8. Data & Resources

Key Database Tables

- Profile: id, display_name, interests[], created_at
- Venue: id, name, latitude, longitude
- Pin: id, user_id, venue_id, status, discoverable
- Match: id, user_a, user_b, venue_id, status
- Message: id, match_id, sender_id, text, created_at

Costs & Budget

- The total budget is **$0.00 for now**. We are leveraging Supabase's free tier, standard Apple Maps on iOS, and existing OpenAI keys.

9. Development Workflow

- **Sprints:** 2-week iterations.
- **Git Process:** Developers work on feature branches and open Pull Requests to main, requiring at least one review.
- **Definition of Done:** Code works on the iOS simulator, passes basic automated checks, and gets approved in PR.

10. Milestone Roadmap

Milestone	Key Deliverables	Expected Output
Milestone 0	Project proposal, GitHub setup, initial backlog, architecture outline	Git tag milestone-0

Milestone 1	MVP: Working Auth, Maps, Check-ins, Matching, and Live Chat	Core loop running in simulator
Milestone 2	Socket.io integration, AI interest ranking, security controls, test suite	Expanded app with automated test coverage
Milestone 3	Automated CI/CD, hosted deployment, video demo, polished documentation	Final **v1.0** tag and release

11. Team Member Breakdown

- **Arda :** Handles React Native views, map rendering, and dark-theme UI components.
- **Margulan :** Manages Supabase backend setup, database migrations, authentication, and Realtime channels.
- **Karthik :** Implements double opt-in matching logic, vector search for interests, and test automation.

12. Risk Management

Risk	Consequence	Plan
Supabase project pauses from inactivity	Backend stops working during grading	Set up an automated ping or wake up project manually prior to submission.
Map fails to load in simulator	Blocks testing	Maintain a fallback venue list view alongside map pins.
Feature creep	Missing deadlines	Stick strictly to the defined MVP features.

13. Success Criteria

1. Running two simulator instances side-by-side completes the full loop (check in → match → send message) in under 60 seconds.
2. Zero external hosting expenses incurred throughout development.
3. Raw location coordinates are never logged or exposed via backend traffic.

14. References

1. Expo Documentation: https://docs.expo.dev/
2. Supabase Realtime Docs: https://supabase.com/docs
3. React Native Maps Repo: https://github.com/react-native-maps/react-native-maps